

COMPUTATIONAL INTELLIGENCE

CASE STUDY

REVIEW - 2

Geophysical Feature Detection

Name	Roll Number
Abhilash B	CB.EN.U4CSE17002
Durga Naga Sai Sravan N	CB.EN.U4CSE17018
Vijay Mani Dev K	CB.EN.U4CSE17031
Sai Akhil Y	CB.EN.U4CSE17050

Abstract :

The rapid improvement of satellite imagery has given us radically improved understanding of our planet. It has enabled us to better achieve everything from mobilizing resources during disasters to monitoring effects of global warming. It also helps us understand the geophysical features of a particular area. Natural features like trees, ponds, waterways, rivers etc., and man-made features like roads, houses, tracks, vehicles etc

Problem Definition:

Every part of geographical area has multiple geo-physical features may it be natural (trees, waterways, rivers, ponds) or man-made (roads, buildings, vehicles etc). The need of identifying these features is eminent as they can provide help in understanding population concentration and helps in mobilizing resources during disasters. So a need arises to implement a model that finds these geo-physical features in a given area of land. We try to use U-Net which is a modified Convolutional Neural Network(CNN) that is mainly useful for image segmentation.

Literature Survey :

1. U-Net: Convolutional Networks for Biomedical Image Segmentation.

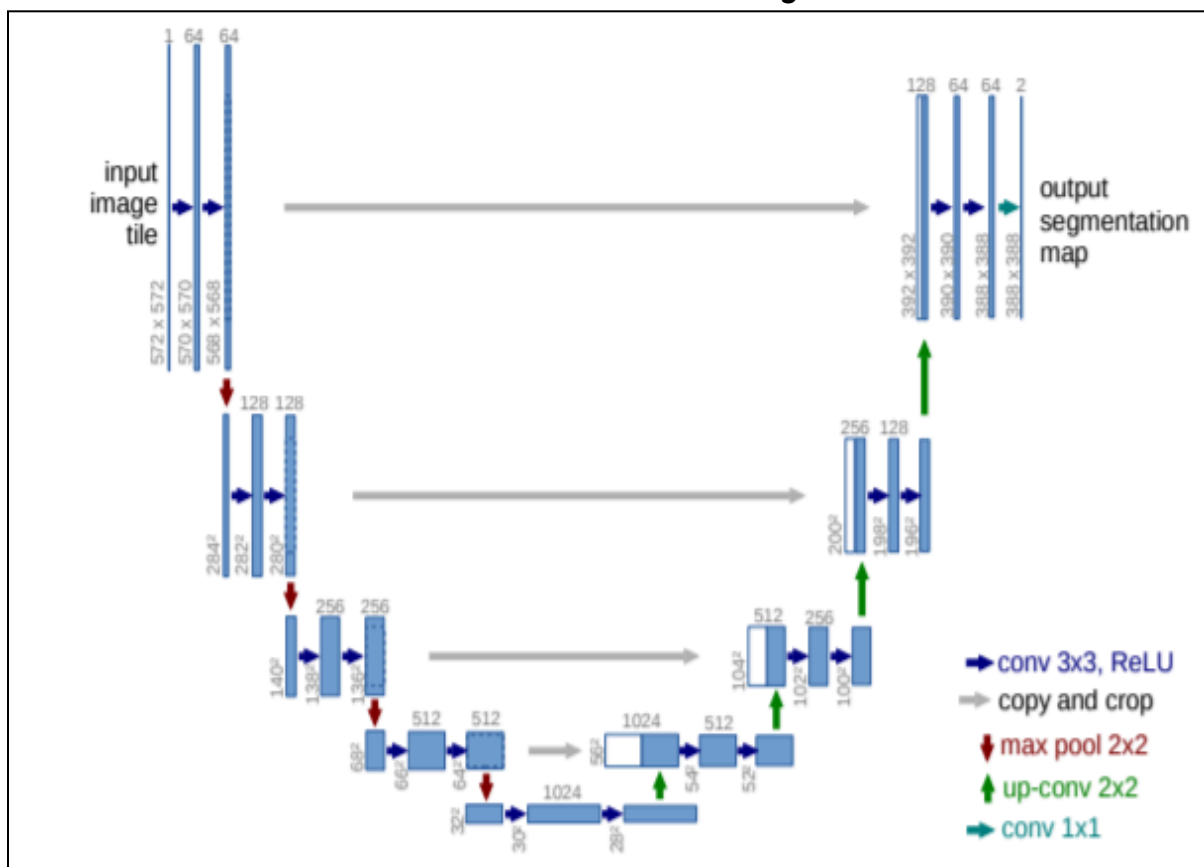
- According to author's view deep convolutional networks have outperformed the state of the art in many visual recognition tasks
- While convolutional networks have already existed for a long time, their success was limited due to the size of the available training sets and the size of the considered networks.
- They showed that such a network can be trained end-to-end from very few images and outperforms the prior best method (a sliding-window convolutional network)
- In this paper, they built a more elegant architecture, the so-called "fully convolutional network". They modified and extended architecture such that it

works with very few training images and yields more precise segmentations.

U-net Architecture

- U-net architecture (example for 32x32 pixels in the lowest resolution).
- Each blue box corresponds to a multi-channel feature map.
- The number of channels is denoted on top of the box.
- The x-y-size is provided at the lower left edge of the box. White boxes represent copied feature maps.
- The arrows denote the different operations.

U-net Architecture Diagram



2.An Introduction to Convolutional Neural Networks - Keiron O'Shea and Ryan Nash

- According to the author CNNs are primarily used to solve difficult image-driven pattern recognition tasks .
- With their precise yet simple architecture, offers a simplified method of getting started with Artificial Neural Network ANNs.

- CNNs are primarily used in the field of pattern recognition within images.
- This allows us to encode image-specific features into the architecture, making the network more suited for image-focused tasks - whilst further reducing the parameters required to set up the model.

Dataset description :

This dataset contains satellite images of 21 classes such as buildings, baseball fields, freeways, etc. The original size of the images is 256x256 pixels. Originally there were 100 images per class. After augmenting each image 4 times the size of each class was brought up to 500 images. This allows for making a more robust mode

The labels of the data are as follows :

Found 7350 images belonging to 21 classes.

{'agricultural': 0, 'airplane': 1, 'baseballdiamond': 2, 'beach': 3, 'buildings': 4, 'chaparral': 5, 'denseresidential': 6, 'forest': 7, 'freeway': 8, 'golfcourse': 9, 'harbor': 10, 'intersection': 11, 'mediumresidential': 12, 'mobilehomepark': 13, 'overpass': 14, 'parkinglot': 15, 'river': 16, 'runway': 17, 'sparseresidential': 18, 'storagetanks': 19, 'tenniscourt': 20}

Sample images



Tools Used :

- Pandas
- Numpy
- Matplotlib
- CNN
- Keras

- Shapely
- Sklearn

Implementation and results:

NN Implementation 1:

CB.EN.U4CSE17031 - Neural Network (NN)

Model :

For classifying images, a particular type of deep neural network, called a convolutional neural network has proved to be particularly powerful. However, modern convolutional neural networks have millions of parameters. Training them from scratch requires a lot of labeled training data and a lot of computing power (hundreds of GPU-hours or more). We only have about 21 labeled photos and want to spend much less time, so we need to be more clever.

Steps:

1) Importing Various Modules:

Numpy for numerical calculations ,Pandas for data management ,Matplotlib for visualization ,tensorflow for model and sklearn for metrics.

2) Preparing the Data

Importing the data by defining the directory of label data.

Setting the fixed dimensions of the images so that input for the model will be in the same size.

Sample code :

```
img_width, img_height = 224, 224

train_data_dir = 'Dataset/train'

validation_data_dir = 'Dataset/test'

if K.image_data_format() == 'channels_first':
```

```
input_shape = (3, img_width, img_height)

else:

    input_shape = (img_width, img_height, 3)
```

3) Modelling

Building the neural network requires configuring the layers of the model, then compiling the model.

Building the ConvNet Model:

Set up the layers

The basic building block of a neural network is the layer. Layers extract representations from the data fed into them. Hopefully, these representations are meaningful for the problem at hand.

Most of deep learning consists of chaining together simple layers. Most layers, such as `tf.keras.layers.Dense`, have parameters that are learned during training.

After the pixels are flattened, the network consists of a sequence of two `tf.keras.layers.Dense` layers. These are densely connected, or fully connected, neural layers. The first Dense layer has 128 nodes (or neurons). The second (and last) layer returns a logits array with length of 10. Each node contains a score that indicates the current image belongs to one of the 10 classes.

Model Summary :

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 222, 222, 32)	896
activation (Activation)	(None, 222, 222, 32)	0
max_pooling2d (MaxPooling2D)	(None, 111, 111, 32)	0
conv2d_1 (Conv2D)	(None, 109, 109, 32)	9248
activation_1 (Activation)	(None, 109, 109, 32)	0
max_pooling2d_1 (MaxPooling2D)	(None, 54, 54, 32)	0
conv2d_2 (Conv2D)	(None, 52, 52, 64)	18496
activation_2 (Activation)	(None, 52, 52, 64)	0
max_pooling2d_2 (MaxPooling2D)	(None, 26, 26, 64)	0
flatten (Flatten)	(None, 43264)	0
dense (Dense)	(None, 64)	2768960
activation_3 (Activation)	(None, 64)	0
dropout (Dropout)	(None, 64)	0
dense_1 (Dense)	(None, 1)	65
activation_4 (Activation)	(None, 1)	0
Total params: 2,797,665		
Trainable params: 2,797,665		
Non-trainable params: 0		

Compile the model

Before the model is ready for training, it needs a few more settings. These are added during the model's compile step:

- Loss function —This measures how accurate the model is during training. You want to minimize this function to "steer" the model in the right direction.
- Optimizer —This is how the model is updated based on the data it sees and its loss function.
- Metrics —Used to monitor the training and testing steps. The following example uses accuracy, the fraction of the images that are correctly classified.

Code :

```
model.compile(loss='categorical_crossentropy',optimizer='rmsprop',
              metrics=['accuracy'])
```

Train the model

Training the neural network model requires the following steps:

1. Feed the training data to the model. In this example, the training data is in the **train_images** and **train_labels** arrays.
2. The model learns to associate images and labels.
3. You ask the model to make predictions about a test set—in this example, the **test_images** array.
4. Verify that the predictions match the labels from the **test_labels** array.

Feed the model

To start training, call the `model.fit` method—so called because it "fits" the model to the training data. No of epochs are set to 3. The training is divided into batch sizes of 10.

```
WARNING:tensorflow:From <ipython-input-11-ct309ac7290>:2: model.fit_generator (from tensorflow.python.keras.engine.training) is
deprecated and will be removed in a future version.
Instructions for updating:
Please use Model.fit, which supports generators.
Epoch 1/3
40/40 [=====] - 7s 167ms/step - loss: 1.1921e-07 - accuracy: 0.6470 - val_loss: 1.1921e-07 - val_accu
acy: 0.8393
Epoch 2/3
40/40 [=====] - 7s 164ms/step - loss: 1.1921e-07 - accuracy: 0.6188 - val_loss: 1.1921e-07 - val_accu
acy: 0.8393
Epoch 3/3
40/40 [=====] - 7s 166ms/step - loss: 1.1921e-07 - accuracy: 0.6074 - val_loss: 1.1921e-07 - val_accu
acy: 0.8393
```

As the model trains, the loss and accuracy metrics are displayed. This model reaches an accuracy of about 0.83 (or 83%) on the training data.

4) Evaluating the Model Performance

Next, compare how the model performs on the test dataset:

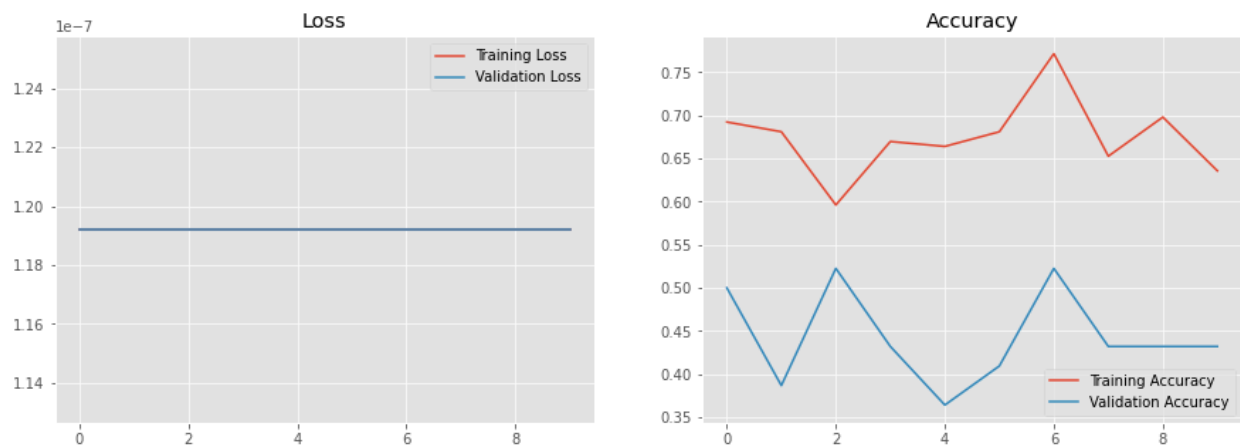
```
test_loss, test_acc = model.evaluate(test_images, test_labels,
verbose=2)
```

```
print('\nTest accuracy:', test_acc)
```

```
313/313 - 1s - loss: 0.1937 - accuracy: 0.7793
```

```
Test accuracy: 0.7793000078201294
```

5) Visualizing Predictions on the Validation Set



We have achieved the accuracy of 77% on the Validation set .

CB.EN.U4CSE17050 -- NN Implementation

Problem Statement:

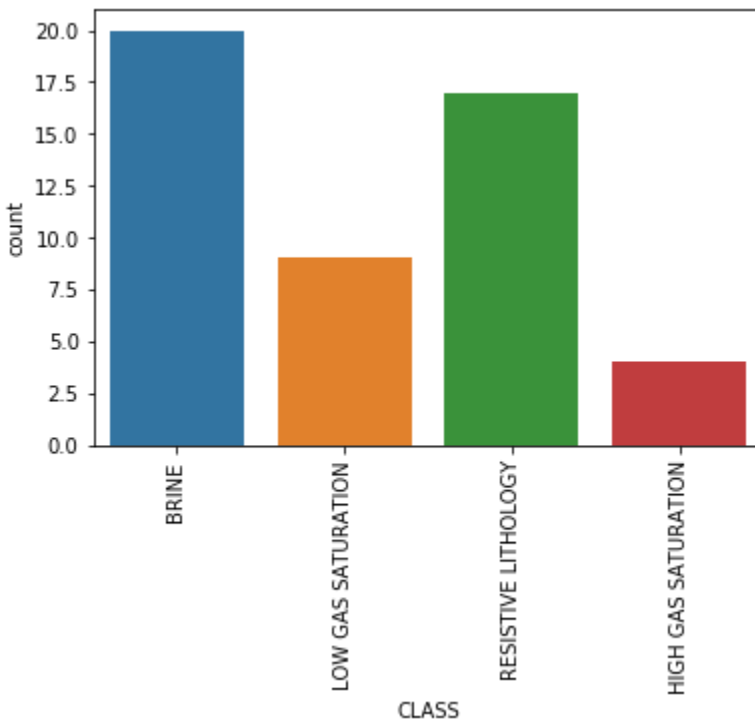
The objective is to classify the given data into gas concentrated, brine or rocky lands based on features like latitude, longitude, gravity, seismic values and also Controlled Source Electromagnetic (CSEM) data recorded at different frequency/offset.

Dataset:

CSEM Data consists of normalized multi-frequency/multi-offset electric fields. This type of data provides information about the resistivity distribution in the Earth. Seismic and gravity responses provide independent and complementary information. If the total multi-physics response is calibrated at wells, using Machine Learning we can create a probabilistic model far away from the wells. In such a way, we can "predict" different scenarios, such as the presence of a reservoir with oil, water or gas in large

exploration area. This dataset was initially released for educational purposes which gives the meaning to its small size which has 50 different data points classified into brine, low gas saturation, high gas saturation and resistive lithology (rocks and barren soil).

REC	CLASS	LONG	LAT	Z(m)	035_OFF1	035_OFF2	035_OFF3	035_OFF4	035_OFF5	...	05_OFF2	05_OFF3	05_OFF4	05_OFF5	05_OFF6	05_OFF7	05_OFF8	05_OFF9	SEISMIC	GRAVITY
0001	ADRIEN	2.438873	3.203038	2173	1.825740	1.676180	1.638216	1.890513	2.058431	...	1.188727	1.208865	1.226899	1.261533	1.384981	1.556937	1.604488	1.617454	0.30	



Model:

Initially 2 baselines models were used to compare the performance with the perceptron model. Gaussian Naive bayes gave the accuracy score of 0.47 which is quite low and Logistic regression gave a score of 0.73 which is quite better than naive bayes model.

Using the Sklearn module ,perceptron model for this dataset is used with max iterations as 200 and random state is specified to make sure the training data get shuffled for each iteration.

Result:

We used test data to predict the classes using the perceptron model specified above. It is seen that perceptron model gives a similar score of 0.73 that is same as the score of logistic regression model

CB.EN.U4CSE17002 -- Fuzzy Logic

Problem Statement

Building a fuzzy logic based system to select 20 students who are eligible for registration fee assistance of 50%. The system reads the input file RegistrationFeeAssistance.csv and outputs a file which contains one column vector containing 20 rows of integer-valued numbers (whole numbers) that represent the ID's (1-100) in the input file.

Dataset Description

RegistrationFeeAssistance.csv (100 rows x 3columns)

Contains data of 100 students with two attributes: Income (Monthly income, real number) and Expenditure (Monthly expenses, real number) of a family and the unit is millions rupees.

And also contains ID's of the students ranging from 1-100.

Approach used:

1) Fuzzification : it is basically the process of converting a crisp input value to a fuzzy value.

i) Determining the membership function:

Degree of membership For Income :

Low Membership (less):

Minimum: 4 ,Maximum: 5

Average Membership (enough):

Lower Minimum: 4 ,Lower Maximum: 5 ,Upper Minimum: 7 ,Upper Maximum: 8

High Membership (more):

Minimum: 7 ,Maximum: 8

	line with positive slope	Flat line	line with negative slope.
Less		$x \leq 4$	$4 < x \leq 5$
Enough	$4 < x < 5$	$5 \leq x \leq 7$	$7 < x \leq 8$
More	$7 < x < 8$	$x \geq 8$	

Degree of membership For Expenditure :

Low Membership (less):

Minimum: 2.5 , Maximum: 5

Average Membership (enough):

Lower Minimum: 2.5 ,Lower Maximum: 5 ,Upper Minimum: 7.5 ,Upper Maximum: 9

High Membership (more):

Minimum: 7.5 , Maximum: 9

	line with positive slope	Flat line	line with negative slope.
Less		$x \leq 2.5$	$2.5 < x \leq 5$

Enough	$2.5 < x < 5$	$5 \leq x \leq 7$.5	$7.5 < x \leq 9$
More	$7.5 < x < 9$	$x \geq 9$	

Then Plotting the Membership function using the membership degree

ii) Creating the Fuzzified Data

```
{'income': {'less': 0, 'enough': 0, 'more': 1},
'expenditure': {'less': 0, 'enough': 0, 'more': 1}}

{'income': {'less': 0, 'enough': 0, 'more': 1},
'expenditure': {'less': 0.192000000000000017, 'enough':
0.80799999999999998, 'more': 0}}
```

2) Rules:

	Income ->	less	enough	more
Expenditure	less	accepted	rejected	rejected
	enough	accepted	consideration	rejected

	more	accepted	accepted	consideration
--	------	----------	----------	---------------

3) Defuzzification:

It is the inversion of fuzzification, here the mapping is done to convert the fuzzy results into crisp results.

Here ,fuzzy set having a group of membership values on the unit interval are reduced to a single scalar quantity.

This method is also alternatively called the "Sugeno defuzzification" method.

(crisp value)

$$X^* = \text{Summation}((\text{inferences}[\text{output}] * \text{deffuzzy}[\text{output}]) / \text{Summation}(\text{inferences}[\text{output}]))$$

Results :

Based on the resulting Crisp Value sorted in descending order with first 20 results as output .

First few results were 98.50, 69.00, 61.66, 51.00, 50.00 ,.....

ID's of the 20 Students who are eligible for registration fee assistance of 50% are

33 , 41 , 25, 75 , 1 , 83 ,38 , 40 , 42 , 43 , 44 ,45 ,46 ,47, 48, 49 , 50 , 52 , 82 , 55 ,56 .

GENETIC ALGORITHM (CB.EN.U4CSE17018)

Problem Statement:

Our primary aim is to use genetic algorithms to enhance the silhouette score for a given data of football players.An enhanced silhouette score provides better clusterization.After

using genetic algorithms,we again use K-means machine learning algorithm for clustering.

Dataset Description:

The dataset contains information about a total of 18000+ players and 86 attributes.It includes latest edition FIFA 2019 players attributes like Age, Nationality, Overall, Potential, Club, Value, Wage, Preferred Foot, International Reputation, Weak Foot, Skill Moves, Work Rate, Position, Jersey Number, Joined, Loaned From, Contract Valid Until, Height, Weight, LS, ST, RS, LW, LF, CF, RF, RW, LAM, CAM, RAM, LM, LCM, CM, RCM, RM, LWB, LDM, CDM, RDM, RWB, LB, LCB, CB, RCB, RB, Crossing, Finishing, Heading, Accuracy, ShortPassing, Volleys, Dribbling, Curve, FKAccuracy, LongPassing, BallControl, Acceleration, SprintSpeed, Agility, Reactions, Balance, ShotPower, Jumping, Stamina, Strength, LongShots, Aggression, Interceptions, Positioning, Vision, Penalties, Composure, Marking, StandingTackle, SlidingTackle, GKDividing, GKHandling, GK Kicking, GKPositioning, GKReflexes, and Release Clause.

Tools Used:

Numpy,pandas,time,matplotlib,sklearn and silhouette score

Implementation:

- Importing all the necessary modules.
- Removing data of goalkeepers and unwanted features in the dataset.Creating a new dataframe with the prepared data.
- Constructing a data matrix from the new prepared data and applying k-means clustering method on it.
- Generating the silhouette score by using euclidean distance as the distance metric.
- Initializing all the required variables and GA parameters such as crossover probability,mutation probability,number of iterations and the pool size.
- We are going to use a random approach in this Genetic algorithm.
- Calculate the fitness of each chromosome.Here we are using the fitness function as Silhouette Coefficient to calculate the score of the model -> [-1, 1].
- While executing the genetic operations,we select the parents randomly (there are several methods: roulette, tournament, rank, random) and we are using a one-point crossover method.
- If the Random value is less than the crossover probability,then we generate children(off-springs) using crossover operator after a random point is selected else if random value is greater than crossover probability then we parent's copy as child.

- In the similar way,if random value is less than the mutation probability,we perform the mutation.
- After each iteration we append the child into the list of offsprings and pool list.
- After successful Implementation of genetic algorithms,we train the initial model(K-means) again.

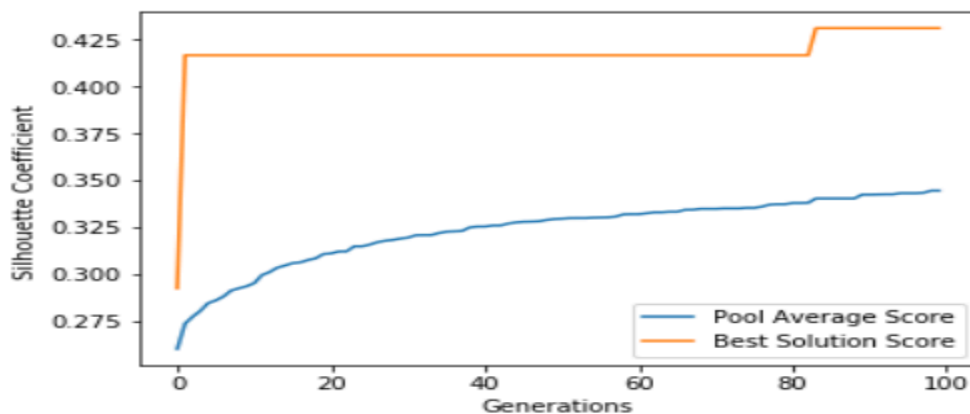
RESULT:

We used genetic algorithms,So the silhouette score increases and hence we will be having an optimized solution.

```
In [10]: score = silhouette_score(X=data_matrix, labels=labels, metric='euclidean')
print(f'\nSilhouette Coefficient:', score)
```

Silhouette Coefficient: 0.22965500741354355

Chosen solution:
['ShortPassing', 'LongShots', 'Marking']
Score: 0.43126676369044387



COMPARISON:

Initial Silhouette Coefficient: 0.22965500741354355

Final Silhouette Coefficient: 0.43126676369044387

CONCLUSION:

In this example we can see how a Genetic Algorithm can be applied to a Machine Learning Algorithm called K-Means. In this Case the Fitness function was implemented using the Silhouette-Coefficient.

Using GA has increased the value of Fitness, so we can say that by using genetic algorithms we can have optimized and better results

